Node.js Fresher 01 - Core Concepts

Field	Value
Level	Fresher
Module	A1 - Node.js Core Concepts (Unit 1, Session 1)
Topic	Overview and glossary, Environment setup, Architecture (Event Loop, V8 Engine, Non-blocking I/O), NPM (package.json, install, scripts), Core modules: path and fs (file handling)
Objective	Understand what Node.js is and how it runs code (V8 + libuv + Event Loop), set up a TypeScript project, manage it with npm, and read/write files safely with the path and fs core modules.
Prerequisite	Basic JavaScript/TypeScript syntax (variables, functions, objects, arrays); basic terminal usage (cd, ls/dir, mkdir).
Assumed version	Node.js 22 LTS or later, TypeScript 5.x, ES Modules ("type": "module")

0. Warm-up - Your first Node.js program

If you have never run Node.js before, do these 3 steps before reading on.

a. Minimum installation

1. Node.js LTS - download from nodejs.org (free) or install through a version manager such as nvm.
2. Editor - Visual Studio Code (free) with the ESLint extension.

Check that the installation succeeded:

```
node -v
# -> v22.x.x (or later)
npm -v
# -> 10.x.x (or later)
```

b. The smallest program

Create a file named hello.ts:

```
const name: string = "Node";
console.log(`Hello, ${name}!`);
```

Run it (tsx runs TypeScript directly, without a separate build step):

```
npx tsx hello.ts
# -> Hello, Node!
```

c. Boilerplate - temporary "magic spells"

Every project in this course starts with the same 3 commands. You do not need to understand them yet; they will be explained in sections 4.2 and 4.4.

```
npm init -y
npm i -D typescript tsx @types/node
npm pkg set type=module
```

You do not need to understand everything yet. Write it, run it, see the result - then everything becomes clear.

1. Problem Context

Anchor

You want to build a small tool called NoteKeeper for your team. Users can save short notes, list them, and read them back. Later the team also wants to see the notes through a tiny web page. It should be simple, run on every laptop (Windows, macOS, Linux), and not need a heavy setup.

Manual

The most natural way for a beginner: write one script, read files with synchronous calls, build file paths by gluing strings together, and copy library files into the project folder by hand.

```
// naive approach
const dir = "C:\\notes\\";              // hard-coded Windows path
const file = dir + title + ".txt";      // string concatenation
const text = readFileSync(file, "utf8"); // waits until the disk answers
```

Breaking point

The tool becomes popular. Then:

- The web page freezes for every user while one big note is being read from disk.
- A teammate on macOS gets "file not found" because the path uses backslashes.
- Another teammate copied a different version of a library - it works on one laptop and crashes on another ("works on my machine").

Core problem: the program waits idly for slow operations (disk, network), file paths and dependencies are managed by hand, and nothing is standardised across machines.

Bridge

We need a runtime that (1) never sits idle while waiting for I/O, (2) ships standard modules for paths and files that work on every OS, and (3) has a package manager that pins library versions. That is exactly what Node.js, its core modules, and npm provide.

2. Old Approach (step-by-step before/after)

A synchronous web server with a hard-coded path, written the "natural" way:

```
import http from "node:http";
import { readFileSync } from "node:fs";

http
  .createServer((req, res) => {
    const file = "C:\\notes\\" + req.url + ".txt"; // OS-specific + unsafe
    const text = readFileSync(file, "utf8");        // blocks the whole server
    res.end(text);
  })
  .listen(3000);
```

Problems:

- Blocking: while readFileSync waits for the disk, no other request can be served.
- Not portable: the path only works on Windows and only on one specific machine.
- Unsafe: req.url comes from the user; a value like ../../secret lets them read other files (path traversal).
- No error handling: a missing file throws an exception and crashes the whole server.
- No dependency management: nothing says which libraries or Node.js version are needed.

3. Weakness

3.1 Blocking calls - one slow task freezes everyone

```
const a = readFileSync("big-1.txt", "utf8"); // 200 ms
const b = readFileSync("big-2.txt", "utf8"); // 200 ms
// 400 ms in which the thread can do nothing else
```

Node.js runs your JavaScript on one main thread. If that thread waits, every other user waits too.

3.2 Hand-built paths - break across operating systems

```
const f1 = "data\\notes.txt";   // works only on Windows
const f2 = "data/" + name;      // double slash if name starts with "/"
```

3.3 Unmanaged dependencies - "works on my machine"

Without package.json and a lock file, every developer installs whatever version they like. Bugs appear on only some machines and are very hard to reproduce.

3.4 Unvalidated input in file paths

Joining user input into a path without checking it allows reading or overwriting files outside the intended folder.

4. Theory - Core concepts of Node.js

4.1 Overview and glossary

Node.js is an open-source, cross-platform JavaScript runtime built on Google's V8 engine. It lets you run JavaScript outside the browser: web servers, command-line tools, scripts, build tools.

Term	Meaning	Example
Runtime	Environment that executes your code and provides APIs	Node.js, Deno, a browser
V8	Google's engine that compiles JavaScript into machine code	Also used in Chrome
libuv	C library behind Node.js that performs async I/O and runs the Event Loop	File, network, timers
Event Loop	Loop that picks finished tasks and runs their callbacks	setTimeout callback
Non-blocking I/O	Start an I/O operation and continue working; get notified when it finishes	fs.promises.readFile
Module	A file or package that exports reusable code	node:fs, ./notes.ts
Core module	Module built into Node.js, no installation needed	node:path, node:fs
npm	Node Package Manager and the public registry of packages	npm install express
package.json	Manifest describing the project, dependencies and scripts	See section 4.4
TypeScript	Typed superset of JavaScript compiled to plain JavaScript	const n: number = 1

Lifecycle of a Node.js process

```
[node app.js] -> [load + run main module] -> [Event Loop while work is pending] ->
[exit]
      |                  |                                  |                           |
   start V8      top-level code runs          timers, I/O, promises run      no work
left,
                 once, top to bottom          as they become ready           process
ends
```

Under the hood - what Node.js is made of

- Where code runs: V8 executes your JavaScript on a single main thread, using a call stack (function calls) and a heap (objects).
- Who does the waiting: libuv asks the operating system to watch sockets and timers (epoll on Linux, kqueue on macOS, IOCP on Windows). File system and some crypto/DNS work run on a libuv thread pool (4 threads by default).
- Order guarantee: your own callbacks always run one at a time on the main thread, so two JavaScript functions never run at exactly the same moment in the same thread.
- Who sees state changes: all code on the main thread shares the same heap, so a variable changed in one callback is visible to the next one.

4.2 Environment setup

1. Install Node.js LTS (nodejs.org or nvm). LTS = Long Term Support: the stable line for production.
2. Create a project folder and run npm init -y. This creates package.json.
3. Install TypeScript tooling as development dependencies: npm i -D typescript tsx @types/node.
4. Add "type": "module" to package.json so that import/export and top-level await work.
5. Optional: npx tsc --init to create tsconfig.json for type-checking.

A minimal tsconfig.json for Node.js with ES Modules:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Ways to run TypeScript:

Command	What it does	Use when
npx tsx src/index.ts	Runs TypeScript directly	Development
npx tsx watch src/index.ts	Re-runs on every file save	Development
npx tsc && node dist/index.js	Compiles to JavaScript, then runs it	Production / CI

Failure cases - environment

Scenario	Symptom	Root cause	Timing
Node.js not on PATH	node: command not found	Installer did not add Node.js to PATH, or terminal was not restarted	Start
Missing "type": "module"	SyntaxError: Cannot use import statement outside a module	File is treated as CommonJS	Run
Wrong Node.js version	Syntax or API not supported	Project needs a newer version than installed	Run
Global install without permission	EACCES error on npm i -g	Writing to a system-owned folder	Install

Scenario	Symptom	Root cause	Timing
Relative import without extension (compiled ESM)	ERR_MODULE_NOT_FOUND	ESM requires the full file name: ./notes.js	Run

4.3 Architecture: Event Loop, V8 Engine, Non-blocking I/O

Node.js handles many tasks with one main thread by never waiting: it starts an operation, registers a callback, and continues. When the operation finishes, the Event Loop runs the callback.

```
Your JS code (V8, main thread)
       |  starts async work (read file, HTTP call, timer)
       v
libuv + OS  ---- do the waiting (thread pool / OS kernel) ----
       |  work finished
       v
Event Loop picks the callback -> runs it on the main thread
```

Event Loop phases (one loop iteration)

Phase	What runs	Typical example
Timers	Callbacks of expired timers	setTimeout, setInterval
Pending callbacks	Deferred system-level callbacks	Some TCP errors
Poll	New I/O events and their callbacks; may wait here	Finished file read, incoming request
Check	setImmediate callbacks	setImmediate
Close callbacks	Cleanup callbacks	socket.on('close')

Between callbacks, Node.js drains two microtask queues: process.nextTick and Promise callbacks (then, await). Microtasks run before the loop moves on.

```
console.log("1 sync");
setTimeout(() => console.log("4 timeout"), 0);
Promise.resolve().then(() => console.log("3 promise"));
process.nextTick(() => console.log("2 nextTick"));
```

Output with ES Modules (this course) - verified with Node.js 26:

```
1 sync
3 promise
2 nextTick
4 timeout
```

Gotcha: in CommonJS the order of promise and nextTick is reversed (nextTick first). In ES Modules the module itself runs inside a promise job, so promise callbacks come first. Never write code that depends on this order.

Blocking vs non-blocking

Aspect	Blocking (sync)	Non-blocking (async)
Example	readFileSync	await readFile (fs/promises)
Main thread while waiting	Stuck, can do nothing	Free to run other callbacks
Code style	Simple, linear	async/await, still reads linearly
Good for	Start-up scripts, CLI tools, config loading	Servers and anything handling many requests

```
import { readFileSync } from "node:fs";
import { readFile } from "node:fs/promises";

// Blocking: nothing else runs until the file is read
const a = readFileSync("a.txt", "utf8");

// Non-blocking: the thread stays free while the OS reads the file
const b = await readFile("a.txt", "utf8");
```

Failure cases - Event Loop

Scenario	Symptom	Root cause	Timing
CPU-heavy loop on main thread	Server stops responding while it runs	Event Loop cannot run other callbacks until the loop ends	Runtime
Forgot await on a promise	Value is a Promise object; code continues too early	Async function returns immediately	Runtime (silent)
Unhandled promise rejection	Process crashes (Node.js 15+)	Rejected promise has no catch / try-catch	Runtime
Endless process.nextTick recursion	Timers and I/O never run	Microtask queue is drained before the loop continues (starvation)	Runtime
Using readFileSync inside a request handler	Latency grows with load	Each call blocks all other requests	Runtime under load

4.4 NPM: package.json, install, scripts

npm installs packages from the npm registry and records them in package.json. A typical file:

```
{
  "name": "notekeeper",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  },
  "dependencies": {},
  "devDependencies": {
    "@types/node": "^22.0.0",
    "tsx": "^4.0.0",
    "typescript": "^5.0.0"
  }
}
```

Field	Purpose
name / version	Project identity (name must be lowercase, no spaces)
type	"module" = ES Modules; omitted = CommonJS
scripts	Named commands run with npm run <name>
dependencies	Packages needed at runtime
devDependencies	Packages needed only during development/build (TypeScript, test tools)
engines	Required Node.js version range (optional)

Essential commands

Command	Effect
npm install	Install everything listed in package.json
npm i <pkg>	Add a runtime dependency

Command	Effect
npm i -D <pkg>	Add a development dependency
npm ci	Clean, exact install from the lock file (use in CI)
npm run <script>	Run a script (npm start and npm test work without "run")
npm uninstall <pkg>	Remove a package
npx <tool>	Run a package binary without installing it globally

Semantic versioning (MAJOR.MINOR.PATCH)

Range	Allows	Example for 1.2.3
^1.2.3	Minor and patch updates	1.2.4, 1.9.0 (not 2.0.0)
~1.2.3	Patch updates only	1.2.4 (not 1.3.0)
1.2.3	Exactly this version	1.2.3

Under the hood - installation and resolution

- npm downloads packages into the node_modules folder next to package.json.
- package-lock.json records the exact version of every package (including nested ones), so every machine installs the same tree.
- When code runs import x from "x", Node.js looks in ./node_modules, then ../node_modules, and so on up the folder tree.

Failure cases - npm

Scenario	Symptom	Root cause	Timing
Lock file not committed	Different versions on different machines	Each install resolves ranges again	Runtime
Typo in script name	npm error Missing script	Script not defined in package.json	Run
Package installed as dependency instead of devDependency (or reverse)	Production build misses a package or ships unnecessary ones	Wrong flag at install time	Deploy
node_modules deleted or missing	Cannot find module 'x'	Dependencies not installed after git clone	Run
Using npm install in CI	Lock file silently changes	npm install may update the lock file; npm ci never does	CI

4.5 Core modules: path and fs (file handling)

Core modules are built into Node.js. Import them with the node: prefix so it is obvious they are not third-party packages.

path - build and parse file paths safely

```
import path from "node:path";

const file = path.join("data", "notes", "todo.txt");
// data/notes/todo.txt (data\notes\todo.txt on Windows)

path.basename(file);          // "todo.txt"
path.basename(file, ".txt");  // "todo"
path.extname(file);           // ".txt"
path.dirname(file);           // "data/notes"
path.resolve("data", "a.txt"); // absolute path based on the current working directory
```

Function	Returns	Note
path.join(...parts)	Parts joined with the correct separator	Result may stay relative
path.resolve(...parts)	Absolute path	Relative parts start from process.cwd()
path.basename(p, ext?)	Last part (file name)	Optional extension is removed
path.extname(p)	File extension	Includes the dot: ".txt"
path.dirname(p)	Folder part	-
import.meta.dirname	Folder of the current file	ES Modules, Node.js 20.11+; replaces __dirname

fs - read and write files

fs offers three styles of API for the same operations:

Style	Import	Behaviour	Use when
Promises	node:fs/promises	Non-blocking, works with await	Default choice
Callbacks	node:fs	Non-blocking, callback(err, data)	Legacy code
Sync	node:fs (...Sync)	Blocking	Start-up scripts and small CLI tools only

```
import { mkdir, writeFile, readFile, readdir, rm } from "node:fs/promises";

await mkdir("data", { recursive: true });         // creates folder if missing
await writeFile("data/note.txt", "Hello", "utf8");
const text = await readFile("data/note.txt", "utf8");
const files = await readdir("data");              // ["note.txt"]
await rm("data/note.txt");
```

Under the hood - how fs/promises works

- await readFile(...) hands the work to libuv, which runs it on the thread pool.
- The main thread is free in the meantime; when the read finishes, the Promise resolves and your code continues after await.
- readFile loads the whole file into memory (a Buffer, or a string if you pass "utf8"). For very large files use streams (createReadStream).

Handling file errors

```
try {
  await readFile("missing.txt", "utf8");
} catch (e) {
  const err = e as NodeJS.ErrnoException;
  if (err.code === "ENOENT") console.error("File not found");
  else throw err;
}
```

Error code	Meaning
ENOENT	File or folder does not exist
EACCES	No permission
EEXIST	File or folder already exists
EISDIR	Expected a file but found a folder
ENOTEMPTY	Folder is not empty (deleting without recursive)

Failure cases - path and fs

Scenario	Symptom	Root cause	Timing
Relative path like "data/a.txt" while running from another folder	ENOENT although the file exists	Relative paths start from process.cwd(), not from the source file	Runtime
User input joined into a path	Files outside the intended folder can be read	../ segments are not filtered (path traversal)	Runtime (security)
writeFile to a folder that does not exist	ENOENT	writeFile does not create folders	Runtime
readFile on a multi-GB file	High memory use or crash	Whole file is loaded into memory	Runtime
Missing await on a promise-based fs call	File not written yet when next line runs; unhandled rejection	Promise not awaited	Runtime (silent)

5. Running Example - NoteKeeper CLI

Throughout this document we build NoteKeeper, a command-line tool that stores each note as a .txt file.

```
notekeeper/
  package.json        <- npm: scripts, dependencies (section 4.4)
  tsconfig.json       <- TypeScript settings (section 4.2)
  src/
    index.ts          <- entry point: reads command-line arguments
    notes.ts          <- uses path + fs/promises (section 4.5)
  data/               <- created at runtime, one .txt file per note
```

Commands:

Command	Result
npm run dev -- add shopping "milk eggs"	Creates data/shopping.txt
npm run dev -- list	Prints the titles of all notes
npm run dev -- read shopping	Prints the content of the note

This example reuses every part of the lesson: the environment, the non-blocking architecture, npm scripts, and the path/fs modules. It will also be used in the Demo and Case Study.

6. Demo - Step by step: from naive script to NoteKeeper

Step 1 - Create the project

```
mkdir notekeeper && cd notekeeper
npm init -y
npm i -D typescript tsx @types/node
npm pkg set type=module
npm pkg set scripts.dev="tsx src/index.ts"
mkdir src
```

Step 2 - notes.ts: path + fs/promises

```
// src/notes.ts
import { mkdir, readdir, readFile, writeFile } from "node:fs/promises";
import path from "node:path";

const DATA_DIR = path.join(import.meta.dirname, "..", "data");

function notePath(title: string): string {
  // basename strips any folder part, the regex removes odd characters
  const safe = path.basename(title).replace(/[^a-z0-9_-]/gi, "-");
  return path.join(DATA_DIR, `${safe}.txt`);
}

export async function addNote(title: string, body: string): Promise<void> {
  await mkdir(DATA_DIR, { recursive: true });
  await writeFile(notePath(title), body, "utf8");
}

export async function listNotes(): Promise<string[]> {
  await mkdir(DATA_DIR, { recursive: true });
  const files = await readdir(DATA_DIR);
  return files
    .filter((f) => path.extname(f) === ".txt")
    .map((f) => path.basename(f, ".txt"));
}

export async function readNote(title: string): Promise<string> {
  return readFile(notePath(title), "utf8");
}
```

Why is this better? The path is built with path.join (portable), it does not depend on the current folder (import.meta.dirname), user input is sanitised, and every call is non-blocking.

Step 3 - index.ts: read command-line arguments

```
// src/index.ts
import { addNote, listNotes, readNote } from "./notes.js";

const [command, title, ...rest] = process.argv.slice(2);

try {
  if (command === "add" && title) {
    await addNote(title, rest.join(" "));
    console.log(`Saved note "${title}"`);
  } else if (command === "list") {
    console.log((await listNotes()).join("\n") || "(no notes)");
  } else if (command === "read" && title) {
    console.log(await readNote(title));
  } else {
    console.log("Usage: add <title> <text> | list | read <title>");
  }
} catch (e) {
  const err = e as NodeJS.ErrnoException;
  console.error(err.code === "ENOENT" ? "Note not found" : err.message);
  process.exitCode = 1;
}
```

Note: the import ends with .js even though the file is notes.ts. This is required by ES Modules (see the failure cases in section 4.2); tsx and tsc both understand it.

Step 4 - Run it

```
npm run dev -- add shopping milk eggs
# -> Saved note "shopping"
npm run dev -- list
# -> shopping
npm run dev -- read shopping
# -> milk eggs
npm run dev -- read unknown
# -> Note not found
```

Step 5 - Prove it is non-blocking

Reading several notes at the same time with Promise.all lets all reads overlap instead of running one after another:

```
const titles = await listNotes();
const bodies = await Promise.all(titles.map((t) => readNote(t)));
console.log(bodies.length, "notes loaded");
```

Failure case - relative path depends on the current folder:

```
// BAD: works only when you start the program from the project root
await readFile("data/shopping.txt", "utf8");

// If you run it from another folder:
// Error: ENOENT: no such file or directory, open 'data/shopping.txt'
```

Runtime trace for "add shopping milk eggs":

```
1. node starts V8, loads index.js, evaluates top-level code
2. process.argv.slice(2) -> ["add", "shopping", "milk", "eggs"]
3. command = "add", title = "shopping", rest = ["milk", "eggs"]
4. addNote("shopping", "milk eggs") starts
5. mkdir(...) is handed to libuv; main thread waits at "await" but is not blocked
6. mkdir finishes -> Promise resolves -> writeFile(...) is handed to libuv
7. writeFile finishes -> console.log("Saved note ...")
8. No work left in the Event Loop -> process exits with code 0
```

7. Self-verification

Run this script to verify the Event Loop order yourself. Save it as order.ts (with "type": "module" in package.json):

```
// order.ts
console.log("1 sync");
setTimeout(() => console.log("4 timeout"), 0);
Promise.resolve().then(() => console.log("3 promise"));
process.nextTick(() => console.log("2 nextTick"));
```

Run: npx tsx order.ts

Expected output (ES Modules):

```
1 sync
3 promise
2 nextTick
4 timeout
```

Experiments:

- Rename the file to order.cjs, remove the type annotations, and run node order.cjs. The order of 2 and 3 changes - why?
- Add a second line setImmediate(() => console.log("5 immediate")). Is it always printed before or after the timeout? Run it several times.

8. Case Study

[Illustrative case - not a real company]

Context: A team builds an internal service that serves HTML report templates. The first version reads the template from disk in every request using readFileSync and builds the path with a string such as "templates\\" + name.

Problems appear:

1. Developers on macOS see "file not found", because the path uses backslashes.
2. During busy hours the service becomes slow for everyone, even for requests that do not need any template.
3. A security review finds that ?name=../../.env can read files outside the templates folder.

Fix, using what this document taught:

1. Build paths with path.join(import.meta.dirname, "templates", path.basename(name)) - portable and safe.
2. Replace readFileSync with await readFile from node:fs/promises so the Event Loop stays free; optionally cache templates in memory after the first read.
3. Handle ENOENT and return a 404 instead of crashing.
4. Add package-lock.json to git and use npm ci in the build pipeline so every machine has the same dependencies.

Result: the same code works on all operating systems, one slow disk read no longer freezes other requests, and the path traversal hole is closed.

9. Common Mistakes - Quick reference

Event Loop and async

Forgetting await:

```
// WRONG - content is a Promise, not a string
const content = readFile("a.txt", "utf8");
console.log(content); // Promise { <pending> }

// RIGHT
const content = await readFile("a.txt", "utf8");
```

Blocking calls in servers:

```
// WRONG inside a request handler
const data = readFileSync("big.json", "utf8");

// RIGHT
const data = await readFile("big.json", "utf8");
```

npm

```
# WRONG - committing installed packages
git add node_modules

# RIGHT - ignore it, commit package.json + package-lock.json
echo node_modules >> .gitignore
```

path and fs

```
// WRONG - OS-specific, string concatenation
const file = "data\\" + name;

// RIGHT
const file = path.join("data", name);
```

```
// WRONG - depends on where you start the program
await readFile("data/a.txt", "utf8");

// RIGHT - relative to this source file
await readFile(path.join(import.meta.dirname, "data", "a.txt"), "utf8");
```

Modules

```
// WRONG in compiled ESM - missing extension
import { addNote } from "./notes";

// RIGHT
import { addNote } from "./notes.js";
```

Summary

Mistake	Symptom	One-line fix
Missing await	Promise { <pending> } or code runs too early	Add await
Sync call in server	Server freezes under load	Use fs/promises
Hand-built path	Fails on another OS	Use path.join
Relative path from cwd	ENOENT from another folder	Use import.meta.dirname
node_modules in git	Huge repository	Add to .gitignore
No "type": "module"	Cannot use import statement outside a module	Set it in package.json
No error handling	Process crashes	try/catch and check err.code

10. Best Practices

10.1 Prefer async APIs; keep the main thread free

Use fs/promises with async/await by default. Use *Sync functions only for start-up code (loading configuration) or small one-off CLI tools.

10.2 When NOT to use - per concept

Concept	Do NOT use when	Consequence	Sign	Better choice
Node.js main thread	Heavy CPU work (image processing, big calculations)	All requests wait	CPU stuck at 100% on one core, slow responses	worker_threads or a separate service
Sync fs functions	Inside request handlers or loops that serve users	Server freezes under load	Latency grows with traffic	fs/promises
readFile	Very large files	High memory usage	Memory spikes	Streams (createReadStream)
New dependency	A few lines of code can do the job	Bigger attack surface, more updates	Package used for one tiny function	Write the function, or use a core module
^ version ranges	Reproducible builds are critical without a lock file	Surprise upgrades	Builds differ between days	Commit lock file and use npm ci

10.3 Trade-off - callbacks, promises and sync calls

Aspect	Sync (readFileSync)	Promises (fs/promises)	Callbacks (fs.readFile)
Pros	Simplest code	Non-blocking, readable with await, easy error handling	Works everywhere, no extra concepts
Cons	Blocks the thread	Must remember await and error handling	Nested code (callback hell), harder error handling
Use when	Start-up, small tools	Default in new code	Old code you maintain

Aspect	Sync (readFileSync)	Promises (fs/promises)	Callbacks (fs.readFile)
Risk	Freezes servers	Unhandled rejections if not awaited	Forgotten error argument

10.4 The one thing most people get wrong

Common misunderstanding: "Node.js is single-threaded, so it is slow and cannot handle many users."

Reality: Only your JavaScript runs on one thread. The waiting for disk and network is done by the OS and libuv, so one Node.js process can keep thousands of connections open. What breaks Node.js is not "one thread" but blocking that thread - with sync calls or long CPU loops. Learning Node.js means learning to keep the Event Loop free.

11. Practice Exercises (run the code, read the output)

Group: Environment and npm

Exercise 1 (Easy): Install Node.js LTS, run node -v and npm -v, and write down the versions. Create hello.ts and run it with npx tsx.

Exercise 2 (Easy): Create a new project with npm init -y. Add the scripts dev, build and start. Run npm run dev.

Exercise 3 (Medium): Install a small package such as dayjs with npm i, and a type-related package with npm i -D. Look at package.json and package-lock.json. Which section did each package go to? Delete node_modules and restore it with npm ci.

Group: Event Loop

Exercise 4 (Easy): Run the order.ts script from section 7. Explain every line of the output.

Exercise 5 (Medium): Predict the output, then run it:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
(async () => {
  console.log("C");
  await null;
  console.log("D");
})();
console.log("E");
```

Exercise 6 (Hard): Write a loop that runs for 3 seconds (while (Date.now() < end) {}). Start a setTimeout of 100 ms before it. When does the timer callback run? Why does it not run after 100 ms?

Group: path

Exercise 7 (Easy): For the path /users/intern/docs/report.final.pdf print basename, extname, dirname and the basename without extension.

Exercise 8 (Medium): Write a function safeJoin(baseDir, userInput) that returns the joined path, but throws an error if the result is outside baseDir (hint: path.resolve and startsWith). Test it with ../secret.txt.

Group: fs

Exercise 9 (Easy): Write a script that creates a folder out, writes 3 .txt files into it, and prints the file names using fs/promises.

Exercise 10 (Medium): Read a file that does not exist. Catch the error and print a friendly message only when err.code is ENOENT; rethrow other errors.

Exercise 11 (Medium): Compare readFileSync and readFile with Promise.all on 5 files of 50 MB each. Measure the time with console.time. Which is faster, and why might the result differ for a single file?

Integration

Exercise 12 (Hard): Extend NoteKeeper with the commands delete <title> and append <title> <text>. Requirements: use fs/promises only, handle ENOENT with a clear message, keep all paths inside the data folder, and add a build script that compiles with tsc and a start script that runs dist/index.js.

12. Recap

What you learned

Concept	Key idea	Keywords / commands
Node.js	JavaScript runtime built on V8 + libuv	node, V8, libuv
Environment	Install LTS, init project, add TypeScript tooling	node -v, npm init, tsx
Event Loop	One main thread runs callbacks when work is ready	timers, poll, check, microtasks
Non-blocking I/O	Start work, continue, resume when finished	async/await, Promise
npm	Install packages and run scripts; lock versions	package.json, npm i, npm ci, npm run
path	Portable path building and parsing	join, resolve, basename, extname
fs	Read and write files; prefer the promise API	readFile, writeFile, readdir, mkdir

Mental model - the restaurant

- Main thread = one waiter. He takes an order, gives it to the kitchen (libuv / OS), and immediately serves the next table.
- Event Loop = the waiter's rule: when the kitchen rings the bell, deliver that dish.
- Blocking = the waiter stands in the kitchen waiting for one dish; every other table waits.
- npm = the supplier list: exactly which ingredients (packages) and which versions.

13. Fresher's Perspective

This lesson teaches the foundation of Node.js. A fresher does not need to know every internal detail - what matters is the mindset: "never block the Event Loop" and "let tools manage paths and dependencies".

What to focus on

1. Async by default: use await with fs/promises; know why sync calls are dangerous in servers.
2. Read error messages: ENOENT, EACCES, ERR_MODULE_NOT_FOUND tell you exactly what is wrong.
3. Commit package.json and package-lock.json, never node_modules.
4. Use path.join and import.meta.dirname instead of string-built paths.
5. Never put user input directly into a file path.

Common fresher mistakes

- Forgetting await and printing Promise { <pending> }.
- Using readFileSync everywhere because it "just works".
- Installing everything globally (npm i -g) instead of per project.
- Ignoring errors - no try/catch around file operations.
- Copying a package into the project folder instead of installing it with npm.

> *Fresher rule: one file = one responsibility; one function = one job; one await = one clear step. Follow these three and your Node.js code will already be better than most beginners'.*